# AI HoloBox

Project Idea: a smart home that acts on its own

Graduation project | AI, Software and Hardware

## 1. The idea

Most of the world's ideas do one of two things: solve a problem, or make something that already exists easier. AI HoloBox does the second. It makes the home, the place where most people spend their comfort time, easier to live in.



Figure 1: the AI HoloBox, a hologram display with a speaker base.

Smart homes already exist, but they mostly wait for a command: "Alexa, turn on the light". AI HoloBox removes the command. The light comes on when you enter a dark room. The AC is already running when you come home hot. The home follows your day without you asking for anything.

### How it knows what to do

- **You enter your data:** your routine and preferences, such as when you wake up and what you want the home to do.
- **Sensors add what is really happening:** if you wake up at 9 instead of 7, the home notices and adapts.
- **You stay in control:** you can change your routine or preferences at any time, because not everyone has a fixed routine.
- **Two ways to talk to it:** the mobile app (chat with the assistant, choose an AI agent and give it your data) and the AI assistant itself.

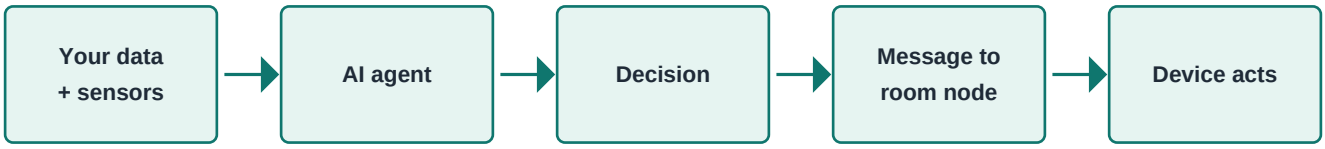



Figure 2: from your data and sensors to a device acting.

# 2. What the graduation version covers

The system is designed in a general way, so it works for any home. For the graduation version it is demonstrated in a **real home, in 2 or 3 rooms**, which is enough to show the whole idea working from data to action.

| Part | Graduation version |
|---|---|
| Hub | One Raspberry Pi 5 that connects everything and runs the decision flow. |
| Room nodes | One ESP32 per room (2 or 3 rooms in the demo). The number of ESP32 boards always equals the number of rooms. |
| App and assistant | A mobile app with a chat to the AI assistant, room entry, and device registration. |
| Safety choice | No mains wiring. The demo uses smart plugs, IR control and low-voltage sensors. |

Each team below follows the same pattern: its **role**, how that role reaches **what the user gets**, and the **stages** the team goes through. The stages are a working draft for the teams to adjust.

# 3. AI section

### Role

The AI team provides the home's decision-maker: an **AI agent**. The agent is given the user's own data (routine and preferences) and receives events from the sensors. From these it decides which device action to take and when, for example turning on the light, setting the AC, or doing nothing.

### How it reaches the user

The user never sees the model. They see the result: a home that does the right thing at the right time, and an assistant they can talk to to change how it behaves. Where the model runs (on the Pi hub or through a service) is decided by the AI team during stage 3.

### Stages

| Stage | What the team does | What it hands over |
|---|---|---|
| 1. Define | List the behaviors the home must show and the device actions the agent may use (on, off, set level, IR command, open/close). | A behavior list and an action list for the other teams. |
| 2. Data | Decide how routines and preferences are written, and prepare example scenarios (late wake-up, coming home hot, a changed routine). | A data format for the app and a test set. |
| 3. Agent prototype | Take an existing AI model and give it the user's data so it acts as the user wants. Make it return device actions in a fixed format. | A working agent that answers with actions. |
| 4. Connect | Connect the agent to sensor events and the device registry, so it knows which devices exist in which room. | An agent that reads real events and sends real actions. |
| 5. Test and refine | Run the scenarios, fix wrong or missing actions, and check it follows a changed routine. | An agent ready for the integrated demo. |

# 4. Software section

## Role

The Software team builds everything the user touches, and the connections that carry information between the user, the AI agent and the hardware.

- **Mobile app:** chat with the AI assistant, choose an AI agent, and give it data on how it should act.
- **Home setup:** the user enters their rooms, and the app shows how many ESP32 room nodes are needed (one per room).
- **Device registration:** the user adds devices themselves by picking a category, a name and a room. An "Add custom device" option covers devices that are not listed yet.
- **Device registry and messaging:** every device and sensor is stored with its room, type and allowed actions, and messages move between the Pi hub and the ESP32 nodes.

## How it reaches the user

The app is the user's window into the system. Whatever the user types or says becomes data for the AI agent, and whatever the agent decides is sent through the software to the right room node.

## Stages

| Stage | What the team does | What it hands over |
|---|---|---|
| 1. Plan | Define the screens and the information each one collects (rooms, devices, routine). | Screen designs and a data list. |
| 2. Core build | Build the app, the chat screen and the storage for rooms, devices and routines. | An app that saves the user's setup. |
| 3. Device registry | Build device categories, custom devices, and the rule that ESP32 count equals room count. | A registry the AI agent can read. |
| 4. Link to AI | Connect the chat and the stored data to the AI agent and show its answers. | A user can talk to the agent from the app. |
| 5. Link to hardware | Send the agent's actions to the room nodes and receive sensor events back. | End-to-end messages between app, hub and nodes. |
| 6. Test | Test adding rooms and devices, changing a routine, and a lost connection. | A stable app for the integrated demo. |

# 5. Hardware section

## Role

The Hardware team builds the physical side: the hub, one node per room, and the sensors and device controllers. It is designed around **categories**, not specific devices, so any new device fits into one of them.

### Device categories (how things are controlled)

| Category | Examples | How it is controlled |
|---|---|---|
| On/off devices | Lights, fan, kettle, heater | Smart plug or relay module |
| Adjustable devices | Light brightness, fan speed | Dimmer or PWM module |
| IR-controlled devices | AC, TV | IR transmitter that copies the remote |
| Motorized devices | Curtains, blinds | Motor driver or servo |
| Already-smart devices | Smart bulbs, smart TV | Wi-Fi, Zigbee or Matter, through the Pi |

### Sensor categories (how the home knows what is happening)

Presence and motion, door and window, temperature and humidity, and light level. Safety sensors (smoke, gas, water leak) can follow later.

### Structure

One **Raspberry Pi 5** is the hub. One **ESP32 per room** reads that room's sensors and drives its devices over Wi-Fi. One ESP32 for a whole house would run out of pins, range and reliability, so each room has its own node.



Figure 3: an ESP32 board in a case, the kind of node placed in each room.

### How it reaches the user

The hardware is how the decisions become real: the light turns on, the AC changes, the curtain moves. It is also how the home notices what you do.

### Stages

| Stage | What the team does | What it hands over |
|---|---|---|
| 1. Choose | Pick the sensors, controllers and ESP32 boards for each category used in the demo. | A parts list. |
| 2. Bench prototype | Build and test one room node on a table: one sensor and one device per category. | A working single node. |
| 3. Hub link | Connect the node to the Pi 5 and exchange messages both ways. | A node that reports and obeys. |
| 4. Build the nodes | Build the remaining 1 or 2 nodes and label them by room. | 2 or 3 room nodes. |
| 5. Install and test | Place nodes in the real rooms using smart plugs, IR and low-voltage sensors, and check range and reliability. | Nodes working inside the home. |

# 6. How the three sections become what the user gets

Each team ends with something the next team needs. The final experience only works when all three connect.

| Step | AI | Software | Hardware |
|---|---|---|---|
| User sets up | Reads the routine and preferences the user entered. | Collects rooms, devices and routine in the app. | Provides one node per room. |
| Something happens | Receives the sensor event (for example: user entered, room is dark). | Passes the event along and keeps the registry current. | Sensors detect it and report. |
| Home responds | Decides the action. | Sends the action to the right room. | The device acts. |
| User changes mind | Follows the changed data from then on. | Lets the user edit data or chat with the assistant. | No change needed. |

### Integrated stages

- **Integration 1:** one room, one sensor, one device, end to end.
- **Integration 2:** the device categories used in the demo, in 2 or 3 rooms.
- **Integration 3:** full scenarios in the real home, including a changed routine and a late wake-up.

### Demo goal

In the real home, the user's routine is entered once. The home then reacts correctly to the user entering a room, coming home, and waking up at a different time, using devices from the categories above, with the user able to change anything from the app or the assistant.